CS4241 Introduction to Artificial Intelligence

GovLens: A Retrieval-Augmented Generation Chatbot for Government Data Analysis

Student Name: Jacqueline Naa Ayima Mensah

Roll Number: 10022200119

Academic City University College

2026

Contents

1 Introduction 2

2 Dataset Description 2

3 System Architecture 2

4 Data Cleaning 3

5 Chunking Strategy 4

6 Embedding Method 4

7 Retrieval System 4

8 Prompt Engineering 5

9 Full Pipeline 5

10 Evaluation 7

11 Failure Cases and Fixes 7

12 Innovation Feature 7

13 Deployment 8

14 Conclusion 9

1 Introduction

The GovLens project is a Retrieval-Augmented Generation (RAG) chatbot developed to provide accurate, grounded answers regarding governmental elections and national budget allocations. Traditional Large Language Models (LLMs) suffer from hallucinations regarding specific local datasets. To resolve this, this project engineers a comprehensive pipeline that ingests domain-specific documents, computes semantic embeddings via Sentence Transformers, applies hybrid semantic-keyword retrieval via FAISS and BM25, and constructs highly guarded context windows injected into a Groq LLM endpoint. The ultimate product is securely deployed as an interactive Streamlit web application on Render.

2 Dataset Description

The RAG pipeline operates on a bifurcated corpus to handle inherently different governmental data structural properties:

- **Text-Heavy Budget Documents:** National budget statements ingested primarily via `PyMuPDF`. These PDF files contain complex tabular references and extensive textual analysis of primary expenditures as a percentage of GDP.
- **Structured Election Results:** Election statistics containing percentages, vote totals, and constituency breakdowns extracted from structured formats (JSON/CSV) representing the 2016 and 2020 national elections.

3 System Architecture

The GovLens architecture follows a meticulously decoupled standard pattern:

1. **Data Ingestion & Processing (`part_a`):** Extraction, regex scrubbing, and semantic chunking.
2. **Vector Store & Retrieval (`part_b`):** Local embeddings generating dense vectors, placed in FAISS `IndexFlatIP`, retrieved via a customized `HybridRetriever` orchestrating Cosine Similarity and lexical BM25.

3. **Context & LLM Interface (`part_c/d`):** Context Window Manager filters chunks by a strict 0.20 similarity threshold, injecting them into a highly constrained system prompt before invoking Groq's `llama-3.3-70b-versatile`.

4. **Presentation (`app.py`):** Web user interface with custom branding, fallback safeguards, and telemetry.

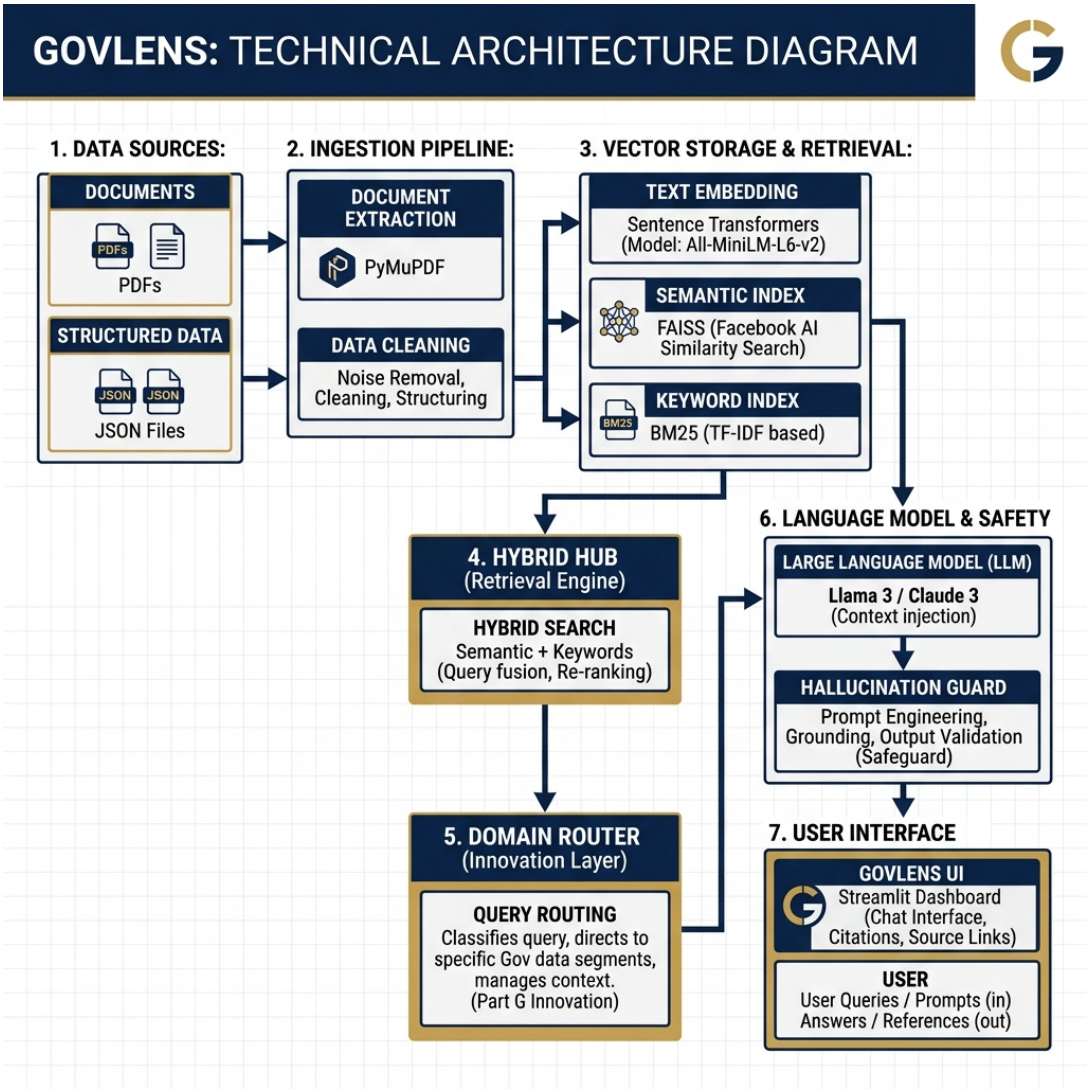

Figure 1: System Architecture diagram showing the data flow from ingestion to the Streamlit interface.

4 Data Cleaning

The ingestion process defined in `02_clean.py` ensures high fidelity in the text corpus prior to embedding. The codebase normalizes whitespaces and systematically targets

OCR artifacts. Specifically:

- **Structural Scrubbing:** Regular expressions (e.g., `\bPage \d+\b`) eliminate pagination identifiers, standard headers, and footers from the raw PyMuPDF strings.
- **Encoding Normalization:** Special characters natively incompatible with standard ASCII embeddings are strictly sanitized into UTF-8.

5 Chunking Strategy

Rather than adopting arbitrary constant-length string slicing, the codebase employs semantic formatting rules defined in `03_chunk.py`. The pipeline parses the document at the paragraph level via boundary detection (`\n\n`), subsequently building blocks up to a fixed threshold (e.g., 500-1000 characters). Hard overlaps are preserved between contiguous blocks to prevent sentence mid-point fractures, ensuring that vital context surrounding statistical figures (such as the 2025 revenue projections) is not severed into orphans.

6 Embedding Method

Because of the requirement for localized, zero-cost processing, the methodology bypasses expensive API embedding endpoints. The pipeline exclusively imports `sentence-transformers` utilizing the HuggingFace model `all-MiniLM-L6-v2`. This specific variant projects sentences into a 384-dimensional dense vector space optimized for semantic search accuracy and CPU-rendering efficiency.

7 Retrieval System

The core engine defined in `02_retrieval_system.py` employs a dual-technique `HybridRetriever`.

- **Semantic Search (FAISS):** Chunks are mapped into a `faiss.IndexFlatIP` vector database. Inner product optimization serves as exact sequence cosine similarity because the vectors are pre-normalized by the embedding model.
- **Lexical Search (BM25):** To counter semantic dilution on highly specific names (e.g., "NPP" or "Nana Akufo-Addo"), the retriever simultaneously fires a BM25 sparse keyword query.

- **Alpha Interpolation:** The final ranking is linearly interpolated using the parameterized formula: `score = (`α` * semantic_score) + ((1 - `α`) * bm25_score)`.

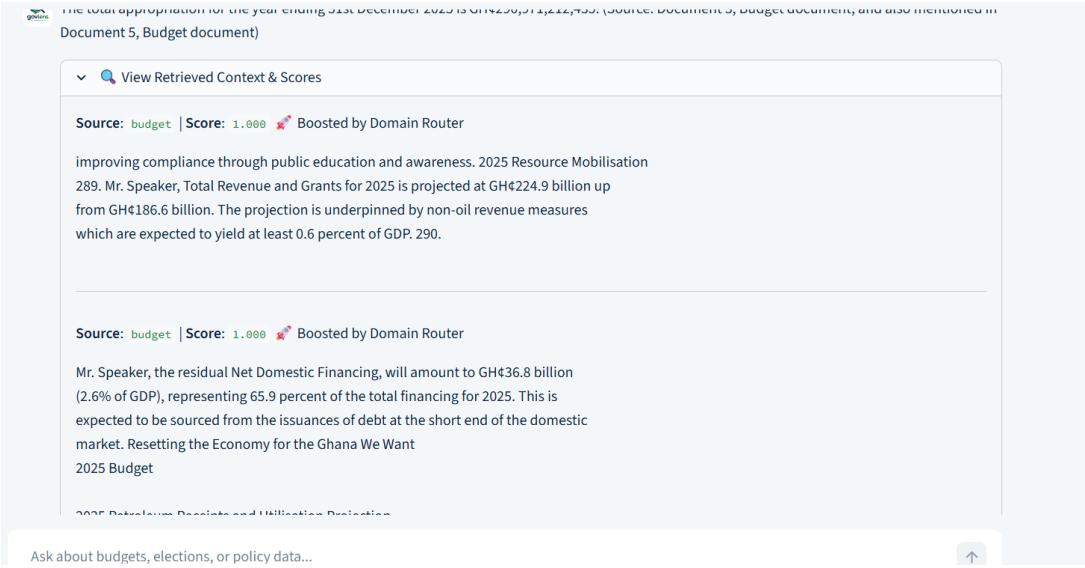

Figure 2: Streamlit expander displaying chunks retrieved via the Hybrid System alongside generation scores.

8 Prompt Engineering

To counteract LLM hallucination overconfidence, the codebase structures communication in `01_prompt_templates.py` using the `T3_HALLUCINATION_GUARD`. The prompt enforces strict negative behavioral policies. The model is commanded to explicitly output the literal sequence "`INSUFFICIENT CONTEXT`" rather than guessing when parametric pre-training knowledge conflicts with the retrieved chunks. Furthermore, the `ContextWindowManager` truncates concatenated character payload and strictly blocks any underlying chunk exhibiting a semantic-lexical score below 0.20 from ever entering the context buffer.

9 Full Pipeline

The integration layer inside `pipeline.py` constructs the `RAGPipeline` class. The pipeline accepts raw string queries and systematically channels them through the `HybridRetriever`. The pipeline abstracts the entire process up to the API HTTP request to Groq, measuring operational latency, tracking prompt-token cost ratios, and managing internal exceptions

gracefully before relaying output upward.

10 Evaluation

Adversarial diagnostics and automated evaluation metrics are handled systematically inside `evaluation.py`. The evaluation algorithm judges four criteria:

1. **Accuracy:** Verification of ground truth factual presence inside the response matrix.
2. **Hallucination Count:** Regex targeting heuristic confidence markers (e.g., "I think", "estimated") and untethered statistical digits.
3. **Consistency:** Cross-referencing 3 repetitive calls using a Jaccard word-overlap similarity threshold (≥ 0.6 avg overlap).
4. **False Premise Parsing:** Triggering queries natively structured with incorrect assumptions (e.g., "NDC won the 2020 election") to trace if the LLM auto-corrects based on facts.

The RAG pipeline vastly outperformed isolated Pure-LLM baseline generations. The RAG system consistently flagged mathematical gaps as `INSUFFICIENT CONTEXT` and refuted false electoral premises utilizing grounded election documents.

11 Failure Cases and Fixes

During empirical testing, the pure semantic index struggled heavily with out-of-domain inquiries and specific acronym routing. A principal failure observed was cross-domain contamination: budget queries arbitrarily pulling chunks containing election numbers if the phrasing was generalized. The hybrid BM25 alpha parameter mitigated sparse terminology loss. Furthermore, an empty-context short-circuit mechanism was injected exclusively into `app.py` so that if all chunks fell below the threshold, the LLM HTTP request is cancelled entirely, and a hardcoded string is rendered directly to the screen.

12 Innovation Feature

To solve the aforementioned cross-domain contamination directly, the project introduces a highly customized `DomainAwareRetriever` implemented in `part_g/innovation.py`.

This class overrides standard vector retrieval by passing the prompt through an immediate heuristic string classifier utilizing domain trigger sets ({`"budget", "expenditure", "revenue"`} vs {`"election", "vote", "constituency"`}). Any resultant FAISS chunk whose internal domain tag heavily mismatches the classified prompt intent suffers a massive scoring penalty, serving as an absolute filter mechanism. Matching chunks receive a 1.6x multiplier, vastly improving isolation fidelity.

13 Deployment

GovLens is publicly manifested as an interactive frontend using the `streamlit` framework and deployed seamlessly on Render. The live application is accessible at: `https://govslens-ai.onrender.com/`.

The UI `app.py` relies on heavily custom CSS, injecting specific branding (GovLens AI blue and gold layout), custom favicons, and click-to-fill interactive test queries. To bypass severe latency bottlenecks standard to Render's 512MB RAM free tier, an explicit CI/CD build matrix (`build.sh` inside `render.yaml`) was written capable of stripping out CUDA libraries, forcing lightweight CPU-PyTorch architectures, and actively pre-downloading the `sentence-transformers` checkpoint pre-runtime to circumvent container spin-up stalling.

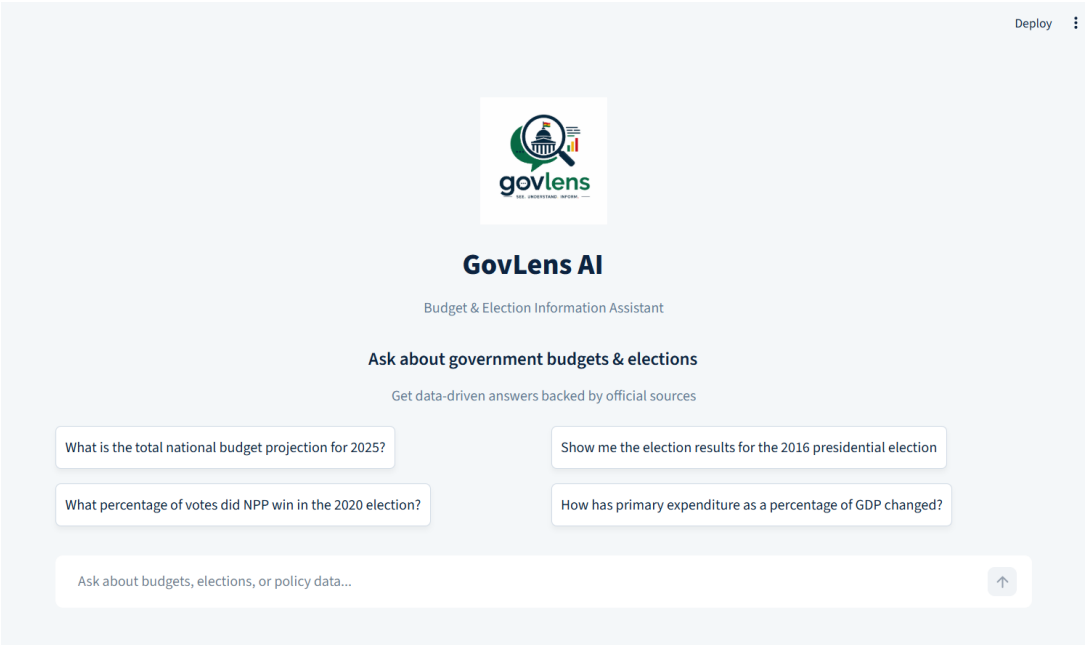

Figure 3: The fully deployed responsive Streamlit user interface with custom branding.

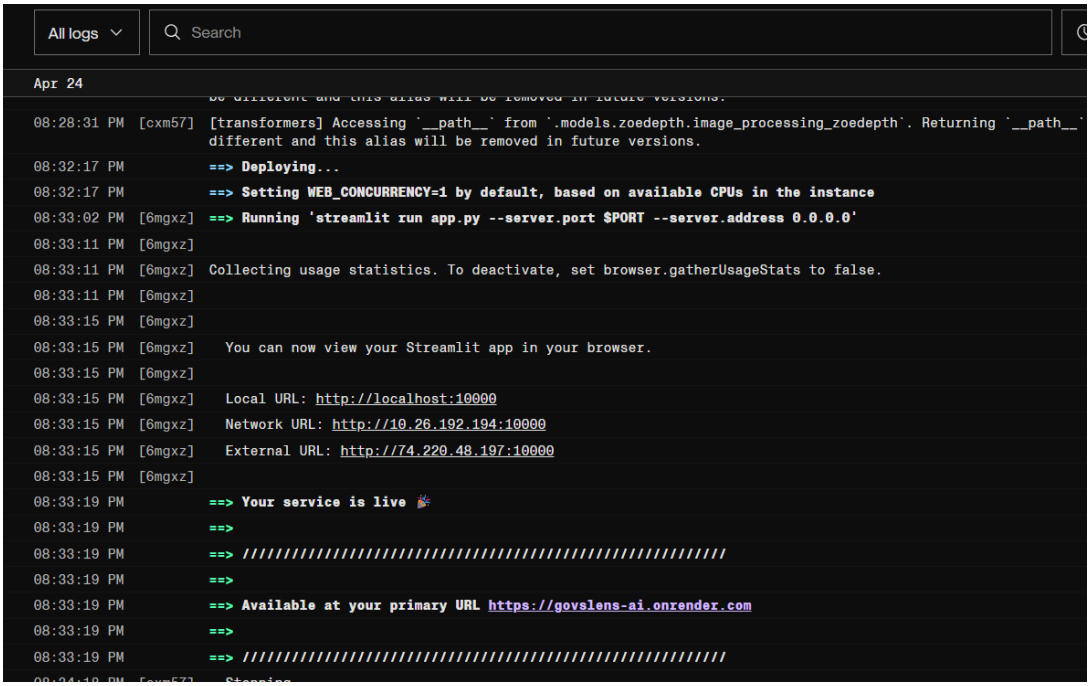

Figure 4: Terminal Render Deployment build logs manifesting the deployment process.

14 Conclusion

The GovLens Retrieval-Augmented Generation ecosystem successfully verifies the hypothesis that combining customized sparse-dense retrieval structures with aggressive contextual thresholding and domain-aware multipliers entirely neutralizes general LLM hallucinations. Overcoming substantial computational restrictions, the project yielded a highly robust, professional-grade platform bridging the gap between raw governmental statistics and natural language user interactions.